Experiment 1: Pulse Code Modulation (PCM)

1. Name of the Experiment

Simulation of Pulse Code Modulation (PCM) and Signal Reconstruction Using Python.

2. Objectives

1. To generate an analog message signal.
2. To perform sampling and quantization.
3. To convert quantized samples into binary codes.
4. To reconstruct the signal and compare it with the original signal.

3. Theory

Pulse Code Modulation (PCM) is a digital modulation technique used to convert an analog signal into digital data.

PCM works in three main steps:

- **Sampling:** Taking samples of the analog signal at regular time intervals.
- **Quantization:** Converting each sample into the nearest available amplitude level.
- **Encoding:** Converting each quantized sample into a binary code.

In this experiment, a sine wave is generated with a message frequency of 5 Hz. The sampling frequency is 100 Hz, and each sample is represented by 4 bits. Therefore, the number of quantization levels is 16.

Finally, linear interpolation is used to reconstruct the signal from the quantized samples.

4. Equations

Original analog signal:

$x(t) = \sin(2\pi fmt)$

Where:

- fm = Message frequency
- t = Time

Sampling interval:

$Ts = 1 / fs$

Number of quantization levels:

$L = 2^n$

Quantization index:

$q = \text{round}[(x + 1)(L - 1) / 2]$

Quantized amplitude:

$xq = 2q / (L - 1) - 1$

PCM bit rate:

$Rb = fs \times n$

Where:

- fs = Sampling frequency
- n = Number of bits per sample

5. Algorithm

1. Start the program.
2. Set the message frequency to 5 Hz.
3. Set the sampling frequency to 100 Hz.
4. Generate the original sine wave.
5. Sample the signal at regular intervals.
6. Calculate the number of quantization levels using $L = 2^n$.
7. Quantize each sample.
8. Convert each quantization index into a 4-bit binary code.
9. Reconstruct the signal using linear interpolation.
10. Plot the original, sampled, quantized, PCM-coded, and reconstructed signals.
11. Stop the program.

6. Numerical Example

Given:

- Message frequency, fm = 5 Hz
- Sampling frequency, fs = 100 Hz
- Number of bits, n = 4
- Duration = 1 second

Step 1: Calculate the number of levels.

$L = 2^4 = 16$ levels

Step 2: Calculate the sampling interval.

Ts = 1 / 100 = 0.01 second

Step 3: Calculate the total number of samples.

Number of samples = fs × Duration

= 100 × 1 = 100 samples

Step 4: Calculate the PCM bit rate.

Rb = 100 × 4 = 400 bits/second

For an input sample x = 1:

q = round[(1 + 1) × 15 / 2] = 15

The binary representation of 15 using 4 bits is **1111**.

Therefore, the maximum-amplitude sample is encoded as 1111.

7. Results & Discussion

The PCM system successfully performs sampling, quantization, and binary encoding. The program generates 100 samples and produces a PCM bit stream of 400 bits.

The plots show the original analog signal, sampled signal, quantized signal, encoded bit stream, and reconstructed signal. The reconstructed signal follows the original signal but may have small differences because of quantization.

Thus, the basic operation of a PCM system is demonstrated successfully using Python.

Experiment 2: Direct Sequence Spread Spectrum (DSSS)

1. Name of the Experiment

Simulation of Direct Sequence Spread Spectrum (DSSS) Using BPSK Modulation.

2. Objectives

1. To generate a binary data sequence.
2. To perform Binary Phase Shift Keying (BPSK) modulation.
3. To spread the data using a pseudo-noise (PN) sequence.
4. To generate and visualize the DSSS signal.

3. Theory

Direct Sequence Spread Spectrum (DSSS) is a communication technique in which the data signal is spread over a wider signal pattern using a PN sequence.

A PN sequence is a predefined sequence of chips used for spreading the data. A chip is a smaller element of the spreading sequence.

In this experiment, the input data is [1, 0, 1]. The binary data is first converted into polar form, where 1 becomes +1 and 0 becomes −1.

Next, the data is multiplied by a PN sequence. The resulting spread signal is multiplied by a cosine carrier to generate the DSSS signal.

The program uses the 5-chip PN sequence [0, 1, 0, 1, 1].

4. Equations

Binary-to-polar conversion:

$d = 2b - 1$

Where b is the input bit.

BPSK signal:

$s(t) = d \cos(2\pi f_c t)$

DSSS signal:

$s_{DSSS}(t) = d \times c(t) \times \cos(2\pi f_c t)$

Where:

- d = Polar data value
- c(t) = PN chip value
- fc = Carrier frequency

Sampling interval:

Ts = 1 / fs

Number of samples per bit:

N = fs × Tb

5. Algorithm

1. Start the program.
2. Set the input bits to [1, 0, 1].
3. Define the PN sequence as [0, 1, 0, 1, 1].
4. Set the sampling frequency to 1000 Hz.
5. Convert the input bits into polar values.
6. Generate the BPSK-modulated signal.
7. Divide each bit duration into five chip intervals.
8. Multiply the polar data by the PN sequence.
9. Multiply the spread signal by the carrier signal.
10. Plot the original bits, BPSK signal, spread signal, and DSSS signal.
11. Stop the program.

6. Numerical Example

Given:

- Input bits = [1, 0, 1]
- PN sequence = [0, 1, 0, 1, 1]
- Bit duration, Tb = 1 second
- Sampling frequency, fs = 1000 Hz
- Carrier frequency, fc = 50 Hz

Step 1: Convert the bits into polar form.

$d = 2b - 1$

Therefore:

$[1, 0, 1] \rightarrow [+1, -1, +1]$

Step 2: Spread the first bit.

The first bit is 1, so its polar value is +1.

Spreading sequence:

$(+1) \times [0, 1, 0, 1, 1]$

$= [0, 1, 0, 1, 1]$

Step 3: Spread the second bit.

The second bit is 0, so its polar value is −1.

$(-1) \times [0, 1, 0, 1, 1]$

$= [0, -1, 0, -1, -1]$

Step 4: Calculate the samples per bit.

$N = 1000 \times 1 = 1000$ samples

For three input bits, the total number of samples is:

$3 \times 1000 = 3000$ samples

7. Results & Discussion

The program generates the original data signal, BPSK signal, spread signal, and DSSS signal. The spreading process uses five chips for each input bit, producing 15 chips in total.

The plots demonstrate how the data is combined with the PN sequence and carrier signal. In this implementation, PN values are 0 and 1, so the signal becomes zero during zero-valued chips.

Thus, the program demonstrates the basic process of DSSS signal generation.

Note: The actual code uses a 5-chip PN sequence, even though one source-code comment refers to seven chips.

Experiment 3: Frequency Hopping Spread Spectrum (FHSS)

1. Name of the Experiment

Simulation of Frequency Hopping Spread Spectrum (FHSS) Using BPSK Modulation.

2. Objectives

1. To generate a binary data sequence.
2. To generate a BPSK-modulated signal.
3. To select different carrier frequencies for different bits.
4. To generate and visualize the FHSS signal.

3. Theory

Frequency Hopping Spread Spectrum (FHSS) is a communication technique in which the carrier frequency changes according to a predefined or generated hopping pattern.

The available carrier frequencies form a set of possible frequencies. During transmission, the system selects one frequency for each bit interval.

In this experiment, the input contains seven bits. Seven possible frequencies are defined: 20, 40, 60, 80, 100, 120, and 140 Hz.

The program selects one frequency for each bit and transmits the BPSK data using the selected frequency. The hopping pattern is generated using a fixed random seed, making the frequency selection reproducible.

4. Equations

BPSK data mapping:

$d = 2b - 1$

BPSK signal:

$sBPSK(t) = d \cos(2\pi fct)$

FHSS signal:

$sFHSS(t) = (2bk - 1) \cos(2\pi fkt)$

Where:

- bk = Input bit at time interval k

- fk = Selected hopping frequency
- t = Time within the bit interval

Number of samples per bit:

$N = fs \times Tb$

5. Algorithm

1. Start the program.
2. Define the input bit sequence.
3. Define the bit duration and sampling frequency.
4. Define the available carrier frequencies.
5. Convert the input bits into polar values.
6. Generate the original data signal.
7. Generate the BPSK-modulated signal.
8. Select one hopping frequency for each bit.
9. Generate the FHSS signal using the selected frequencies.
10. Plot the original bits, BPSK signal, hopping pattern, and FHSS signal.
11. Stop the program.

6. Numerical Example

Given:

- Input bits = [1, 0, 1, 1, 0, 1, 0]
- Bit duration, Tb = 1 second
- Sampling frequency, fs = 1000 Hz
- Available frequencies = [20, 40, 60, 80, 100, 120, 140] Hz

For the fixed random seed used in the program, the selected hopping sequence is:

[100, 40, 140, 80, 80, 100, 40] Hz

Step 1: Consider the first bit.

The first bit is 1 and the selected frequency is 100 Hz.

Therefore:

$s1(t) = \cos(2\pi \times 100 \times t)$

Step 2: Consider the second bit.

The second bit is 0 and the selected frequency is 40 Hz.

Since 0 is mapped to −1:

$s2(t) = -\cos(2\pi \times 40 \times t)$

Step 3: Calculate the samples per bit.

$N = fs \times Tb$

$= 1000 \times 1 = 1000$ samples

Step 4: Calculate the total samples.

Total samples = 7 × 1000 = 7000 samples

7. Results & Discussion

The program successfully generates the original data signal, BPSK signal, frequency-hopping pattern, and FHSS signal.

The carrier frequency changes from one bit interval to another according to the generated hopping sequence. The program shows that different carrier frequencies can be used to transmit the data.

Thus, the basic operation of FHSS signal generation is demonstrated using Python.

Experiment 4: Convolutional Coding, BPSK, AWGN, and Viterbi Decoding

1. Name of the Experiment

Simulation of Convolutional Coding with BPSK Modulation, AWGN Channel, Viterbi Decoding, and BER Performance Analysis.

2. Objectives

1. To encode binary data using convolutional coding.
2. To modulate the encoded bits using BPSK.
3. To transmit the signal through an Additive White Gaussian Noise (AWGN) channel.
4. To decode the received data using the Viterbi algorithm.
5. To calculate the Bit Error Rate (BER).
6. To compare coded and uncoded BPSK performance at different SNR values.

3. Theory

Convolutional coding is a channel-coding technique that adds extra bits to the original data. These extra bits help the receiver detect and correct errors.

In this experiment, a convolutional encoder with code rate 1/2 and constraint length K = 3 is used. The generator polynomials are (7, 5) in octal form.

After encoding, the bits are modulated using BPSK. In BPSK, binary 0 is represented by −1 and binary 1 is represented by +1.

The signal passes through an AWGN channel, which adds Gaussian noise to the transmitted signal. A hard-decision detector converts the received signal into binary bits.

Finally, the Viterbi algorithm finds the most likely input sequence by comparing possible state transitions and selecting the path with the lowest Hamming distance.

The BER is calculated for different Eb/N0 values. Both coded and uncoded BPSK systems are compared.

4. Equations

Code rate:

$R = k / n$

For this experiment:

$R = 1 / 2 = 0.5$

Convolutional encoder outputs:

$o1 = (b + s1 + s2) \bmod 2$

$o2 = (b + s2) \bmod 2$

Next state:

Next state = 2b + s1

Where:

- b = Input bit
- s1 and s2 = Previous state bits
- o1 and o2 = Encoded output bits

BPSK mapping:

$x = 2b - 1$

Conversion from dB to linear Eb/N0:

Eb/N0 (linear) = 10^[Eb/N0 (dB) / 10]

Noise standard deviation for coded transmission:

$\sigma = \sqrt{[1 / (2R \times Eb/N0)]}$

Bit Error Rate:

BER = Number of bit errors / Total transmitted information bits

5. Algorithm

1. Start the program.
2. Generate a binary message.
3. Initialize the encoder state to 00.
4. Encode the message using the generator polynomials (7, 5).
5. Add two tail zeros to return the encoder to state 00.
6. Convert the encoded bits into BPSK symbols.
7. Generate Gaussian noise.
8. Add noise to the transmitted signal using the AWGN channel.
9. Apply hard-decision detection to the received signal.
10. Use the Viterbi algorithm to decode the received bits.
11. Remove the two tail bits from the decoded message.

12. Count the bit errors and calculate BER.
13. Repeat the experiment for different Eb/N0 values from 0 dB to 7 dB.
14. Plot the received signals, decoded data, trellis path, and BER curves.
15. Compare coded and uncoded system performance.
16. Stop the program.

6. Numerical Example

Given:

- Input message = [1, 1, 0]
- Code rate, R = 1/2
- Constraint length, K = 3
- Eb/N0 = 4 dB

Step 1: Add tail bits.

The original message is:

[1, 1, 0]

Adding two tail zeros gives:

[1, 1, 0, 0, 0]

Step 2: Perform convolutional encoding.

Using the generator polynomials (7, 5), the encoded output is:

1101011100

There are five input bits, including the two tail bits, and ten output bits.

Step 3: Perform BPSK modulation.

Using 0 → −1 and 1 → +1:

Encoded bits:

[1, 1, 0, 1, 0, 1, 1, 1, 0, 0]

BPSK symbols:

[+1, +1, −1, +1, −1, +1, +1, +1, −1, −1]

Step 4: Convert Eb/N0 into linear form.

Eb/N0 = 10^(4/10)

≈ 2.512

Step 5: Calculate the noise standard deviation.

$\sigma = \sqrt{[1 / (2 \times 0.5 \times 2.512)]}$

$\sigma \approx 0.631$

Therefore, the noise standard deviation for this coded transmission is approximately 0.631.

Step 6: Decode the received signal.

The received signal is converted into bits using hard-decision detection. The Viterbi decoder then selects the most likely bit sequence.

In the absence of decoding errors, the recovered information message is:

[1, 1, 0]

7. Results & Discussion

The program demonstrates convolutional encoding, BPSK modulation, AWGN channel transmission, hard-decision detection, and Viterbi decoding.

The trellis diagram shows the possible state transitions and the path selected by the Viterbi decoder. The BER graph compares the coded and uncoded systems over Eb/N0 values from 0 dB to 7 dB.

As Eb/N0 increases, the noise becomes less significant compared with the signal energy, so BER is generally expected to decrease. Convolutional coding can improve error performance by allowing the decoder to use the redundancy in the encoded bits.

The exact BER values are calculated and printed when the program runs. They should be taken from the actual program output rather than assumed.

Thus, the experiment demonstrates the working of a convolutional coding system and its error-correction performance.